CONDUIT CONTROL CENTER

Software Requirements Specification

Version 1.1 — Raspberry Pi Edition

Version 1.1
May 31, 2026
Supersedes: SRS v1.0
Status: Draft

Raspberry Pi 4 · Ubuntu 22.04 ARM64 · MIT Licence
FastAPI · Nginx · Python 3 · Vanilla JS

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the Conduit Control Center (CCC) — a lightweight, open-source web dashboard for managing a Psiphon Conduit node on a Raspberry Pi 4. It is the authoritative reference for developers, contributors, and users of the project.

1.2 What Changed in v1.1

This revision of the SRS incorporates the following deliberate simplifications compared to v1.0:

Change	Rationale
MIT licence replaces GPLv3	More permissive; reduces contributor friction; aligns with the broader Python/web ecosystem.
JWT tokens moved to v1.2+	Cookie-based sessions are simpler, secure enough for a single-user dashboard, and require no key management.
TOTP / 2FA moved to v1.3+	Adds setup complexity for beginners. Can be retrofitted cleanly when there is demand.
RS256 key pair removed	Not needed without JWT. Eliminates a complex installation step.
Token blacklisting removed	Server-side session store provides equivalent revocation without a separate system.
RBAC deferred to v1.2	Single-user dashboard has no need for roles in v1.x.
MVP scope (v0.1) added	Defines a shippable first release that contributors can build in weeks, not months.
GitHub governance section added	Provides clear contribution rules from day one.

1.3 Project Overview

Conduit Control Center is a self-hosted management dashboard that replaces complex Psiphon Conduit CLI commands with a clean, beginner-friendly web interface. It is designed to run comfortably on a Raspberry Pi 4 with Ubuntu 22.04 ARM64 and to be installed in under 15 minutes by someone learning Linux.

> ℹ Design philosophy: Start simple. Ship something real. Grow through community feedback.

1.4 Definitions

Term	Definition
Conduit	Psiphon software that lets users share bandwidth to help others bypass internet censorship.
CCC	Conduit Control Center — this project.
Conduit CLI	The official Psiphon command-line tool for managing a Conduit node.
FastAPI	A modern Python web framework for building REST APIs.
Nginx	A lightweight web server and reverse proxy.
Session cookie	An HttpOnly, Secure, SameSite=Strict browser cookie that identifies an authenticated session.
MVP	Minimum Viable Product — the smallest working version worth releasing.
SRS	Software Requirements Specification.
ARM64	64-bit ARM architecture used by Raspberry Pi 4.
TOTP	Time-based One-Time Password — a common 2FA mechanism (planned for v1.3+).
JWT	JSON Web Token — a signed authentication token (planned for v1.2+).

2. System Overview

2.1 Architecture

The architecture is deliberately minimal. No build toolchain, no Node.js runtime, no database server, and no message broker are required. The entire stack runs inside a Python virtual environment.

Layer	Component	Technology
Presentation	Web Dashboard	HTML5 / CSS3 / Vanilla JavaScript (no build step)
API	Backend REST API	Python 3.10+ / FastAPI
Adapter	Conduit Adapter	Python module wrapping Conduit CLI
Execution	Conduit CLI	Official Psiphon binary (ARM64)
Reverse Proxy	TLS termination & routing	Nginx
DNS & TLS	Domain & certificate	Cloudflare Origin Cert (recommended) or Let's Encrypt
DDNS	Dynamic DNS A-record updates	Cloudflare API + cloudflare-ddns.sh (cron, 5 min)
State	Metrics & sessions	SQLite (no separate server needed)

2.2 Request Flow

```
Browser  →  Cloudflare Edge (HTTPS/TLS 1.3, proxied)
         →  Nginx (HTTP, Cloudflare origin pull, port 80 or 8080)
         →  FastAPI (127.0.0.1:8000)
         →  Conduit Adapter  →  Conduit CLI  →  Psiphon Network
```

2.3 Target Hardware

Specification	Value
Device	Raspberry Pi 4 Model B (4 GB RAM recommended; 2 GB minimum)
OS	Ubuntu 22.04 LTS ARM64 (also tested: Raspberry Pi OS 64-bit)
CPU	Broadcom BCM2711 Quad-core Cortex-A72 @ 1.8 GHz
Storage	MicroSD 16 GB minimum / USB SSD recommended for reliability
Network	Gigabit Ethernet with a public IP or Cloudflare tunnel

2.4 Cloudflare Deployment Architecture

Cloudflare is the primary and recommended deployment model for Conduit Control Center. All public deployments are assumed to use Cloudflare unless the user explicitly chooses the direct-IP alternative.

Concern	Primary (Cloudflare)	Secondary (Direct IP)
DNS	Cloudflare-managed DNS (Proxy enabled, orange cloud)	Any DNS provider
TLS termination	Cloudflare edge (TLS 1.3 to browser); Origin Cert on Nginx	Nginx with Let's Encrypt cert
SSL/TLS mode	Full (strict) in Cloudflare dashboard	N/A
Dynamic IP	cloudflare-ddns.sh via Cloudflare API (cron every 5 min)	Manual or third-party DDNS
Real visitor IP	CF-Connecting-IP header; restored by Nginx real_ip module	$remote_addr directly
DDoS / WAF	Cloudflare proxy layer	None (user responsibility)
Self-signed cert	Not permitted for public deployment	Not permitted for public deployment

ℹ Because Cloudflare proxy is enabled, Nginx never sees the browser directly. All connections arrive from Cloudflare edge IPs. Nginx MUST use ngx_http_realip_module with the CF-Connecting-IP header to restore the real visitor IP for logging and rate limiting.

Pre-Installation Checklist

The install script validates each item below via the Cloudflare API at the start of setup. If any item is missing, the script exits immediately with a plain-English explanation rather than failing silently mid-install.

Pre-install task	Who does it	Validated by install.sh?
Domain added to Cloudflare	User (Cloudflare dashboard)	Yes — zone lookup
DNS A record created for dashboard hostname	User (Cloudflare dashboard)	Yes — record lookup
Cloudflare Proxy enabled on record (orange cloud)	User (Cloudflare dashboard)	Yes — proxied: true check
SSL/TLS mode set to Full (strict)	User (Cloudflare dashboard)	No — user confirms verbally
Origin Certificate created and saved to Pi	User (Cloudflare dashboard)	Yes — openssl verify

Pre-install task	Who does it	Validated by install.sh?
API token created with Zone:DNS:Edit + Zone:Zone:Read	User (Cloudflare dashboard)	Yes — API call test
curl and jq installed on Pi	install.sh (automated)	N/A — installed by script

⚠ SSL/TLS mode (Full strict) cannot be verified via the API without a higher-permission token. The install script will ask the user to confirm this step is done before proceeding, and will print the exact Cloudflare dashboard path.

Cloudflare DDNS Component

The Raspberry Pi 4 is deployed on a home internet connection with a dynamic public IP. The cloudflare-ddns.sh script updates the Cloudflare A record for the configured hostname every 5 minutes via cron.

The project uses Script B behaviour: the script reads the current proxy status from the Cloudflare API and preserves it when updating the IP. This means the user can toggle the orange/grey cloud in the Cloudflare dashboard at any time without the DDNS script overriding their choice. Script A behaviour (forcing a specific proxy status) is not used because it would conflict with the user's manual Cloudflare settings.

Required secrets for the DDNS component (stored in /etc/conduit-cc/.env):

- CF_API_TOKEN — Cloudflare API token with Zone:DNS:Edit permission for the target zone
- CF_ZONE_NAME — The root domain (e.g. rockysystem.net)
- CF_RECORD_NAME — The full hostname to update (e.g. conduit.rockysystem.net)

⚠ CF_API_TOKEN is a high-value secret. It must never appear in source code, logs, or any file tracked by git. It is loaded exclusively from /etc/conduit-cc/.env (permissions: 640, owned by conduit-cc).

3. MVP Scope — Version 0.1

⚠ Version 0.1 is the first public release. It must be useful and installable by a beginner. Features not listed here are explicitly out of scope for v0.1.

3.1 Goals of v0.1

Version 0.1 has one job: let a Raspberry Pi owner install the dashboard, see their Conduit node running, and control it without touching the terminal again. Everything else can wait.

3.2 In Scope for v0.1

#	Feature	Acceptance Criteria
1	Install script (install.sh)	Runs on fresh Ubuntu 22.04 ARM64; completes without errors; prints dashboard URL.
2	Password-based login	Login page accepts correct password; rejects wrong password; sets session cookie.
3	Logout	Clears session cookie; redirects to login page.
4	Node status display	Dashboard shows Running / Stopped / Error with last-updated timestamp.
5	Start / Stop / Restart	Buttons trigger correct Conduit CLI actions; UI reflects new state within 5 seconds.
6	Basic traffic counter	Shows total bytes transferred today (up/down) from Conduit metrics.
7	System health panel	Shows CPU %, RAM %, CPU temperature, disk usage. Updates every 10 seconds.
8	Log viewer	Displays last 200 lines of Conduit log. Refreshes every 30 seconds.
9	HTTPS via Cloudflare (primary) or Let's Encrypt (secondary)	Primary: Cloudflare Proxy + Origin Certificate + Full (strict). Secondary: Let's Encrypt. Self-signed: local testing only, never public.
10	Change password	Settings page allows password change after entering current password.
11	DDNS status panel	Dashboard shows last DDNS update time, current public IP, and last update result (success/fail).

3.3 Explicitly Out of Scope for v0.1

- JWT tokens — plain secure session cookies are used instead.
- TOTP / two-factor authentication

- Let's Encrypt auto-renewal via Certbot — supported for direct-IP deployments; documented in docs/tls-setup.md but not automated by install.sh in v0.1
- Metrics persistence / time-series charts
- Email or webhook alerting
- Multi-node support
- Dark mode
- Conduit configuration editor (read-only config view is acceptable)
- Backup and restore

3.4 Definition of Done for v0.1

Version 0.1 is considered done when all of the following are true:

1. All 10 in-scope features pass their acceptance criteria on a real Raspberry Pi 4.
2. The install.sh script runs successfully on a fresh Ubuntu 22.04 ARM64 image.
3. Unit test coverage is at or above 60% for backend modules.
4. README.md contains working install instructions and a screenshot.
5. The GitHub repository passes all CI checks (lint, test, security audit).
6. A CHANGELOG.md entry exists for v0.1.0.

4. Functional Requirements

This section covers the full functional requirements across all planned versions. Each requirement is tagged with the version in which it is targeted.

4.1 Authentication & Session Management

FR-AUTH-01: Local Admin Account [v0.1]

The system SHALL support one local administrator account created during installation. The password SHALL be hashed with bcrypt (cost factor 12 minimum). No default password SHALL be set; the install script SHALL prompt the user to create one.

FR-AUTH-02: Login [v0.1]

The login page SHALL accept a username and password. On success, a secure server-side session SHALL be created and a session cookie SHALL be issued (HttpOnly, Secure, SameSite=Strict). On failure, a generic error message SHALL be shown without revealing whether the username or password was incorrect.

FR-AUTH-03: Logout [v0.1]

Clicking logout SHALL invalidate the server-side session and clear the session cookie. The user SHALL be redirected to the login page.

FR-AUTH-04: Account Lockout [v0.1]

After 5 consecutive failed login attempts, the account SHALL be locked for 15 minutes. A CLI command (ccc-unlock) SHALL allow the administrator to unlock the account from the terminal.

FR-AUTH-05: Session Expiry [v0.1]

Idle sessions SHALL expire after 60 minutes (configurable). On expiry the user SHALL be redirected to the login page with a friendly message.

FR-AUTH-06: JWT Authentication [v1.2]

Future: JWT Bearer token support for programmatic API access. Session cookies will remain the default for the web UI.

Note: TOTP / two-factor authentication is planned for v1.3.

4.2 Conduit Node Management

FR-NODE-01: Start / Stop / Restart [v0.1]

The dashboard SHALL provide clearly labelled buttons to start, stop, and restart the Conduit service via systemd. Stop and Restart SHALL require a confirmation dialog. Each action SHALL display a spinner while in progress.

FR-NODE-02: Node Status [v0.1]

The dashboard SHALL display the real-time Conduit node status: Running, Stopped, Starting, Stopping, or Error. Status SHALL be polled every 5 seconds. The time of the last status change SHALL be shown.

FR-NODE-03: Pairing Workflow [v0.1]

The dashboard SHALL provide a step-by-step pairing guide. The pairing link entered by the user SHALL be used transiently in memory only. It SHALL NOT be stored, logged, or persisted anywhere.

FR-NODE-04: Configuration Viewer [v0.1]

The dashboard SHALL display the current Conduit configuration in read-only mode. Sensitive values (tokens, keys) SHALL be masked.

FR-NODE-05: Configuration Editor [v1.1]

The dashboard SHALL allow editing of non-sensitive Conduit configuration fields via a validated web form. Changes SHALL prompt for a service restart.

FR-NODE-06: Update Notification [v1.1]

The dashboard SHALL check for Conduit CLI updates on page load and display a notification when a newer version is available.

4.3 Metrics & Monitoring

FR-METRICS-01: System Health Panel [v0.1]

The dashboard SHALL display: CPU usage (%), RAM usage (MB and %), CPU temperature (°C), and disk usage (GB and %). Metrics SHALL update every 10 seconds using a lightweight polling API call.

FR-METRICS-02: Basic Traffic Counters [v0.1]

The dashboard SHALL show total bytes transferred (upload and download) since the Conduit service last started.

FR-METRICS-03: DDNS Status [v0.1]

The dashboard SHALL display a DDNS status panel showing: current public IP, last successful update timestamp, last update result (success or failure with error message), and the configured hostname. This data is read from the DDNS script log file at /var/log/conduit-cc/ddns.log.

FR-METRICS-04: Time-Series Charts [v1.1]

The dashboard SHALL display time-series charts for traffic and system health with configurable windows (1h, 24h, 7d). Charts SHALL be rendered in the browser using a lightweight library (Chart.js).

FR-METRICS-05: Metrics Persistence [v1.1]

Metrics SHALL be stored in SQLite and retained for 30 days. Older data SHALL be automatically purged by a background job.

FR-METRICS-06: Threshold Alerts [v1.1]

Dashboard alerts SHALL be shown when: CPU temperature > 80°C, RAM usage > 90%, disk usage > 85%, or the Conduit service crashes.

4.4 Log Management

FR-LOGS-01: Log Viewer [v0.1]

The dashboard SHALL display the last 200 lines of the Conduit service log, refreshing every 30 seconds. A manual refresh button SHALL also be provided.

FR-LOGS-02: Log Level Filter [v1.1]

The log viewer SHALL support filtering by log level (INFO, WARNING, ERROR) and free-text search.

FR-LOGS-03: Log Export [v1.1]

The user SHALL be able to download logs as a .tar.gz archive filtered by date range.

FR-LOGS-04: Sensitive Data Redaction [v0.1]

Pairing links SHALL never appear in logs. The log viewer SHALL redact any string matching a known pairing link pattern before display.

4.5 Settings

FR-SETTINGS-01: Change Password [v0.1]

The administrator SHALL be able to change their password from a settings page. The current password SHALL be required for confirmation.

FR-SETTINGS-02: Session Timeout [v0.1]

The administrator SHALL be able to configure the session idle timeout (default: 60 minutes).

FR-SETTINGS-03: Alert Thresholds [v1.1]

The administrator SHALL be able to configure the CPU temperature, RAM, and disk usage thresholds that trigger dashboard alerts.

FR-SETTINGS-04: Backup & Restore [v1.2]

Future: export and import a configuration backup archive (secrets excluded).

4.6 REST API

FR-API-01: OpenAPI Documentation [v0.1]

All API endpoints SHALL be documented via the auto-generated FastAPI OpenAPI interface at /api/docs. This endpoint SHALL require authentication.

FR-API-02: Core Endpoints [v0.1]

Endpoint	Purpose
POST /api/auth/login	Authenticate and create session
POST /api/auth/logout	Invalidate session
GET /api/status	Node and system status

Endpoint	Purpose
POST /api/conduit/{start|stop|restart}	Control Conduit service
GET /api/metrics/system	CPU, RAM, disk, temperature
GET /api/metrics/traffic	Bytes transferred counters
GET /api/logs	Last N lines of Conduit log
GET /api/health	Health check (unauthenticated)
GET /api/ddns/status	Last DDNS update time, current public IP, last result
GET/PUT /api/settings	Read/write application settings

5. Non-Functional Requirements

5.1 Performance

All targets assume a Raspberry Pi 4 with 4 GB RAM running Ubuntu 22.04.

ID	Requirement	Target
NFR-P-01	API response time (GET endpoints)	< 300 ms at p95
NFR-P-02	Dashboard page load	< 3 seconds on LAN
NFR-P-03	CCC idle CPU overhead	< 5% of one core
NFR-P-04	CCC total RAM usage	< 150 MB
NFR-P-05	App + dependency disk footprint	< 500 MB

5.2 Reliability

- NFR-R-01: The application SHALL be managed by systemd with Restart=on-failure and RestartSec=5.
- NFR-R-02: The application SHALL start automatically on boot.
- NFR-R-03: The dashboard SHALL remain partially functional (read-only status) even if the metrics collection job fails.
- NFR-R-04: Configuration SHALL not be lost on unexpected shutdown (atomic file writes).

5.3 Usability — Raspberry Pi Friendly

✓ This project prioritises beginners. Every design decision should ask: "Can a Raspberry Pi hobbyist follow this?"

- NFR-U-01: A user with basic Linux knowledge SHALL complete installation in under 15 minutes using install.sh.
- NFR-U-02: Every user action SHALL show visible confirmation or feedback within 2 seconds.
- NFR-U-03: The dashboard SHALL be usable on screens 768px wide and above.
- NFR-U-04: All forms SHALL show inline validation with plain-language error messages (no jargon).
- NFR-U-05: The install script SHALL detect and report common failure conditions with a human-readable fix suggestion.
- NFR-U-06: Error pages SHALL include a link to the project's GitHub Discussions page for help.

5.4 Maintainability

- NFR-M-01: Backend code SHALL follow PEP 8 and be linted with ruff.
- NFR-M-02: Frontend code SHALL use no build toolchain; plain HTML, CSS, and JavaScript only.
- NFR-M-03: Unit test coverage SHALL be at or above 60% for v0.1 and 75% for v1.x releases.
- NFR-M-04: A CHANGELOG.md SHALL be updated for every release following Keep a Changelog conventions.
- NFR-M-05: All Python dependencies SHALL be pinned with exact versions in requirements.txt.

5.5 Compatibility

- NFR-C-01: The backend SHALL support Python 3.10 and above.
- NFR-C-02: The frontend SHALL work on the latest release of Chrome, Firefox, Safari, and Edge.
- NFR-C-03: The system SHALL work on both Raspberry Pi 4 ARM64 and standard x86-64 Ubuntu servers.
- NFR-C-04: API responses SHALL remain backwards-compatible within a major version.

6. Security Requirements

Security requirements are calibrated for a single-user self-hosted dashboard on a hobbyist device, not a multi-tenant enterprise product. The goal is solid baseline security that does not require a security engineering background to configure.

ℹ Authentication model for v0.1–v1.1: bcrypt password hash + secure server-side session cookie. JWT tokens are introduced in v1.2 for API access only. TOTP is introduced in v1.3 as an optional enhancement.

6.1 Authentication

- SEC-AA-01: All routes and endpoints SHALL require an authenticated session except /login and /api/health.
- SEC-AA-02: Passwords SHALL be hashed with bcrypt, cost factor 12 minimum.
- SEC-AA-03: Session cookies SHALL be HttpOnly, Secure, and SameSite=Strict.
- SEC-AA-04: Sessions SHALL be stored server-side (SQLite). Session IDs SHALL be cryptographically random (32 bytes).

6.2 Transport Security

ID	Requirement
SEC-TLS-01	All traffic SHALL be served over HTTPS. HTTP SHALL redirect to HTTPS with a 301 response.
SEC-TLS-02	Nginx SHALL be configured for TLS 1.2 minimum, TLS 1.3 preferred.
SEC-TLS-03	HSTS header SHALL be sent with max-age of at least 31536000 seconds.
SEC-TLS-04	Cloudflare deployments (recommended): use Cloudflare Proxy + a Cloudflare Origin Certificate installed on Nginx + SSL mode set to Full (strict) in the Cloudflare dashboard.
SEC-TLS-05	Non-Cloudflare deployments: use Let's Encrypt via Certbot. The install script SHALL include Certbot setup instructions. Auto-renewal SHALL be configured via a systemd timer or cron job.
SEC-TLS-06	Self-signed certificates SHALL NOT be used for any public-facing deployment. They are permitted only for local development and testing. The install script SHALL warn the user if a self-signed certificate is detected on a public IP.

6.3 Input Validation & Common Vulnerabilities

ID	Requirement
SEC-INP-01	All API inputs SHALL be validated by Pydantic models. Invalid inputs SHALL return HTTP 422 with a non-leaking error.

ID	Requirement
SEC-INP-02	All database queries SHALL use parameterised statements to prevent SQL injection.
SEC-INP-03	All HTML output SHALL be context-escaped to prevent XSS.
SEC-INP-04	CSRF protection (double-submit cookie pattern) SHALL be applied to all state-changing requests.

6.4 Secret Management

ID	Requirement
SEC-SEC-01	No secrets SHALL be stored in source code or committed to version control.
SEC-SEC-02	Secrets SHALL be loaded from a .env file at /etc/conduit-cc/.env, excluded from git via .gitignore.
SEC-SEC-03	The repository SHALL include .env.example with placeholder values and inline documentation.
SEC-SEC-04	Pairing links SHALL be processed in memory only and SHALL NEVER be logged or stored.
SEC-SEC-05	Config file permissions: 640 for config files, owned by conduit-cc service user.

6.5 Network Hardening

ID	Requirement
SEC-NET-01	FastAPI SHALL bind to 127.0.0.1 only. Nginx is the sole internet-facing process.
SEC-NET-02	Nginx SHALL rate-limit the login endpoint to 10 requests/second per IP.
SEC-NET-02a	When Cloudflare proxy is enabled, Nginx MUST use ngx_http_realip_module: set real_ip_header CF-Connecting-IP and set_real_ip_from for all current Cloudflare IP ranges. Rate limiting MUST operate on the restored real IP, not the Cloudflare edge IP.
SEC-NET-03	Security headers: Content-Security-Policy, X-Frame-Options: DENY, X-Content-Type-Options: nosniff.
SEC-NET-04	install.sh SHALL configure UFW to allow only ports 22, 80, and 443.

6.6 Audit Logging

- SEC-AUDIT-01: All login attempts (success and failure) SHALL be logged with timestamp and source IP.

- SEC-AUDIT-02: All Conduit service control actions SHALL be logged with timestamp.
- SEC-AUDIT-03: Audit log entries SHALL NOT contain passwords, session IDs, or pairing links.

7. Deployment Requirements

7.1 Install Script

DR-INST-01: install.sh

A single shell script SHALL automate the complete installation on Ubuntu 22.04 ARM64. The script SHALL be idempotent. If a step fails, the script SHALL print a clear error message and exit gracefully without leaving the system in a broken state.

DR-INST-02: Installation Steps

7. Verify Ubuntu 22.04 and ARM64 architecture. Print a friendly error and exit if the check fails.
8. Update apt and install: python3, python3-pip, python3-venv, nginx, certbot, ufw, curl.
9. Create system user conduit-cc with no home directory and no login shell.
10. Copy application files to /opt/conduit-cc/.
11. Create Python virtual environment at /opt/conduit-cc/venv/ and install dependencies.
12. Generate a 32-byte random session secret and write it to /etc/conduit-cc/.env.
13. Prompt for an admin password, hash it with bcrypt, write to /etc/conduit-cc/config.json.
14. Install systemd service unit, reload daemon, enable and start the service.
15. Configure Nginx virtual host. Test config and reload Nginx.
16. Configure UFW: allow 22, 80, 443; enable if not already active.
17. Install cloudflare-ddns.sh to /opt/conduit-cc/scripts/; set permissions 750; install cron job: */5 * * * * conduit-cc /opt/conduit-cc/scripts/cloudflare-ddns.sh >> /var/log/conduit-cc/ddns.log 2>&1
18. Print the dashboard URL and post-install checklist.

✓ The install script should feel welcoming, not scary. Use clear progress messages and plain English.

7.2 Service Management

Requirement	Detail
Service name	conduit-cc.service
Run as user	conduit-cc (not root)
Start on boot	systemctl enable conduit-cc
Restart policy	Restart=on-failure, RestartSec=5
Sandboxing	ProtectSystem=strict, PrivateTmp=yes, NoNewPrivileges=yes
Logs	journalctl -u conduit-cc -f

7.3 File Layout

```
/opt/conduit-cc/          Application files and virtual environment
/etc/conduit-cc/          Runtime config, secrets (.env), session database
/etc/systemd/system/      conduit-cc.service
/etc/nginx/sites-available/  conduit-cc.conf
/var/log/conduit-cc/      Application logs
```

7.4 Upgrade & Health Check

- DR-UPG-01: An update.sh script SHALL upgrade the application in-place with service downtime under 10 seconds.
- DR-UPG-02: The upgrade script SHALL back up /etc/conduit-cc/ before making any changes.
- DR-HEALTH-01: GET /api/health SHALL return HTTP 200 JSON with {version, uptime, status}. No authentication required.

8. Open-Source Requirements

8.1 Licence — MIT

ℹ This project uses the MIT Licence. It is short, permissive, and widely understood. It imposes no viral obligations on contributors or users.

- OSS-LIC-01: The project SHALL be released under the MIT Licence.
- OSS-LIC-02: A LICENSE file containing the full MIT licence text SHALL be present in the repository root.
- OSS-LIC-03: Every source file SHALL include the SPDX identifier on line 1: # SPDX-License-Identifier: MIT
- OSS-LIC-04: Third-party licences SHALL be listed in a NOTICES file.

```
MIT License

Copyright (c) 2026 Conduit Control Center Contributors

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software are
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND...
```

8.2 Repository Structure

```
conduit-control-center/
  ├── README.md               Project overview, install instructions, screenshot
  ├── LICENSE                 MIT licence text
  ├── CHANGELOG.md            Version history
  ├── CONTRIBUTING.md         How to contribute
  ├── SECURITY.md             Vulnerability disclosure policy
  ├── CODE_OF_CONDUCT.md      Contributor Covenant
  ├── NOTICES                 Third-party licence attributions
  ├── .env.example            Template for required secrets
  ├── config.example.json     Template for application config
  ├── install.sh              Installation script
  ├── uninstall.sh            Removal script
  ├── update.sh               In-place upgrade script
  ├── backend/                FastAPI application
  ├── frontend/               HTML, CSS, JavaScript (no build step)
  ├── scripts/                Utility scripts
  ├── tests/                  Unit and integration tests
  └── docs/                   Architecture, API reference, guides
```

8.3 Documentation Requirements

- OSS-DOC-01: README.md SHALL include: project description, feature list, install instructions, a screenshot, a licence badge, and a link to CONTRIBUTING.md.
- OSS-DOC-02: CONTRIBUTING.md SHALL cover: dev environment setup, how to run tests, coding standards, commit message format, and pull request process.
- OSS-DOC-03: docs/ SHALL contain an architecture diagram, API reference, and a deployment guide written for beginners.

9. GitHub Governance

This section defines how the project is managed on GitHub. Clear governance from day one helps contributors know what is expected and makes the project easier to maintain as it grows.

9.1 Branch Strategy

Branch	Purpose
main	Always releasable. Direct pushes are disabled. Changes enter via pull request only.
develop	Integration branch for the next release. Feature branches are merged here first.
feature/<name>	New feature work. Branched from develop; merged back via pull request.
fix/<name>	Bug fixes. Branched from develop (or main for hotfixes).
docs/<name>	Documentation-only changes.

9.2 Pull Request Rules

- GOV-PR-01: All changes to main SHALL be made through a pull request. Direct pushes are disabled.
- GOV-PR-02: Every pull request SHALL require at least one approving review before merging.
- GOV-PR-03: CI checks (lint, tests, security audit) SHALL pass before a pull request can be merged.
- GOV-PR-04: Pull requests SHALL include a description of what changed, why, and how it was tested.
- GOV-PR-05: Pull requests that fix a bug SHALL include a regression test.
- GOV-PR-06: Stale pull requests (no activity for 30 days) SHALL be labelled stale and closed after a further 14 days if inactive.

9.3 Issue Management

Issue Type	Handling
Bug Report	Use the Bug Report template. Must include: steps to reproduce, expected vs actual behaviour, OS/version.
Feature Request	Use the Feature Request template. Describe the user need, not the implementation.
Security Vulnerability	Do NOT open a public issue. Email the address in SECURITY.md. Response expected within 7 days.
Question / Help	Use GitHub Discussions, not Issues.

9.4 Release Process

19. All features and fixes for the release are merged into develop and tested.
20. CHANGELOG.md is updated with a summary of changes under the new version heading.
21. A release branch release/vX.Y.Z is created from develop.
22. A pull request from the release branch to main is opened for final review.
23. On merge, a Git tag vX.Y.Z is created and a GitHub Release is published.
24. The release notes in GitHub SHALL link to the CHANGELOG entry.

9.5 Versioning

The project follows Semantic Versioning (semver.org):

- MAJOR: Breaking API or behaviour change.
- MINOR: New functionality, backwards compatible.
- PATCH: Bug fixes, backwards compatible.

Note: Pre-release versions (0.x) may have breaking changes between minor versions. This is expected and documented in the CHANGELOG.

9.6 Code of Conduct

- GOV-COC-01: A CODE_OF_CONDUCT.md based on the Contributor Covenant v2.1 SHALL be included.
- GOV-COC-02: The README SHALL link to the Code of Conduct.
- GOV-COC-03: Code of conduct reports SHALL be handled privately by the project maintainer.

9.7 Continuous Integration

CI Check	Tool / Command
Linting	ruff check .
Unit tests	pytest tests/unit/
Integration tests	pytest tests/integration/
Security audit	pip-audit
Coverage report	pytest --cov=backend --cov-report=xml
Install script syntax	bash -n install.sh

ℹ CI runs on every pull request and every push to main and develop. A failed CI check blocks merging.

10. Future Roadmap

The roadmap below shows the planned progression of features beyond v0.1. Scope is intentionally kept small per version so that each release ships quickly and community feedback can shape the direction.

Version	Theme & Key Features	Notable Additions
v0.1	MVP — Ship it	Login, node control, status, basic metrics, log viewer, HTTPS, install script
v1.0	Stable foundation	Configuration editor, time-series charts, threshold alerts, full Certbot/Cloudflare cert automation, full test coverage
v1.1	Developer experience	Dark mode, log filters, metrics export (CSV), update notifications, i18n (Farsi + Chinese)
v1.2	API & automation	JWT token authentication for API, backup/restore, API key management, webhook alerts
v1.3	Security hardening	TOTP / 2FA (optional), session revocation UI, audit log viewer
v2.0	Multi-node	Manage multiple Conduit nodes from one dashboard, RBAC, Prometheus /metrics endpoint, Docker image

⚠ Features are added to a version based on community feedback and contributor availability. This roadmap is a guide, not a contract.

11. Constraints & Assumptions

11.1 Constraints

- The application MUST run within 150 MB of RAM to leave headroom for Conduit and the OS.
- The application MUST NOT require a build step for the frontend (no webpack, Vite, TypeScript compiler).
- The application MUST NOT require a database server. SQLite is the maximum database dependency.
- The application MUST NOT require cloud services, telemetry, or external APIs in its core functionality.
- The install script MUST produce a working system on a fresh Ubuntu 22.04 ARM64 image without manual intervention.

11.2 Assumptions

- The user has SSH access to the Raspberry Pi and can run sudo commands.
- The Raspberry Pi has a stable public IP or a dynamic DNS / Cloudflare tunnel pointing to it.
- The Psiphon Conduit CLI binary is available for ARM64 Ubuntu.
- The user has accepted the Psiphon Conduit Terms of Service independently.
- The primary deployment uses Cloudflare-managed DNS with proxy enabled. The Cloudflare API token (CF_API_TOKEN) has Zone:DNS:Edit permission for the target zone.
- The Raspberry Pi 4 is on a home internet connection with a dynamic public IP. The cloudflare-ddns.sh script is the mechanism for keeping the DNS A record current.
- Let's Encrypt is the supported alternative for users who deploy without the Cloudflare proxy.

12. Revision History

Version	Date	Author	Description
1.0	2026-05-31	Kiavash	Initial SRS draft
1.1	2026-05-31	Kiavash	Simplified for Raspberry Pi: MIT licence, session auth, MVP v0.1 scope, GitHub governance added

END OF DOCUMENT